



Faculty of Engineering and Materials Science
German University in Cairo

Smart Task & Quality Management System

A thesis submitted in partial fulfilment of the requirements for the degree of
Bachelor of Science in Computer Science and Engineering

By

Ahmed Hassan Abdelbarr

Supervised by

Wael Zakaria Abdallah

May 1, 2026

This is to certify that:

- (i) the thesis comprises only my original work toward the Bachelor Degree of Science (B.Sc.) at the German University in Cairo (GUC),
- (ii) due acknowledgment has been made in the text to all other material used

Ahmed Hassan Abdelbarr
May 1, 2026

Acknowledgments

I would like to express my sincere gratitude to my supervisor, **Associate Professor Wael Zakaria Abdallah**, for his guidance, constructive feedback, and continuous support throughout the development of this project and the writing of this thesis.

I am grateful to the Faculty of Engineering and Materials Science at the German University in Cairo for providing a stimulating academic environment and the resources necessary to carry out this work.

My heartfelt thanks go to my family for their unwavering encouragement and patience throughout my studies.

Finally, I extend my appreciation to the open-source communities behind the technologies used in this project—including Microsoft .NET, React, Electron, MongoDB, and the Tailwind CSS team—whose publicly available documentation and tooling made this work possible.

List of Publications

No publications arose from this work at the time of submission.

Abstract

Modern knowledge workers increasingly depend on digital task management tools to organise work, coordinate with teams, and meet deadlines. However, most contemporary solutions are cloud-centric, rendering them unavailable in low-connectivity environments, and none provide a tightly integrated AI assistant that understands the full project context.

This thesis presents *Task Flow*, an offline-first desktop task and quality management application built with a hybrid architecture. The system pairs a local SQLite database for primary persistence with a MongoDB relay layer for cross-device data synchronisation when connectivity is available. The backend is implemented in ASP.NET Core (.NET 10) using a repository pattern, Entity Framework Core, JWT authentication, and SignalR for real-time notifications. The frontend is built with React 19 and TypeScript, packaged as a cross-platform desktop application via Electron 41.

Task Flow provides five distinct task visualisation modes—list, Kanban board, table, Gantt chart, and calendar—alongside project management, team collaboration, real-time direct and group messaging, and a calendar with event scheduling. A key contribution is the embedded AI assistant powered by the Mistral API, offering three specialised modes: a general productivity assistant, a code helper, and a document scanner using optical character recognition. The assistant streams responses incrementally using Server-Sent Events, providing a responsive conversational experience.

An outbox-pattern offline synchronisation engine ensures that operations performed without network access are reliably replayed once connectivity is restored, with progress broadcast to the client via SignalR. The application is distributed as a self-contained installer for Windows, macOS, and Linux using NSIS, DMG, and AppImage packaging respectively, with the .NET runtime embedded so no prior installation is required.

Evaluation demonstrates that Task Flow successfully delivers a full-featured, offline-capable task management environment with integrated AI assistance, addressing a clear gap in the existing landscape of productivity desktop software.

Keywords: task management, offline-first, SQLite, MongoDB, ASP.NET Core, React, Elec-

tron, AI assistant, Mistral, real-time synchronisation, SignalR.

Contents

Acknowledgments	iii
List of Publications	iv
List of Abbreviations	x
List of Figures	xi
List of Tables	xii
1 Introduction	xiii
1.1 Background and Motivation	xiii
1.2 Problem Statement	xiv
1.3 Aims and Objectives	xiv
1.4 Scope and Limitations	xv
1.5 Thesis Overview	xvi
2 Literature Review	xviii
2.1 Task Management and Productivity Software	xviii
2.2 Desktop Application Frameworks	xix
2.3 Offline-First Architecture	xx
2.4 Hybrid Database Architectures	xxi
2.5 Real-Time Communication Patterns	xxii
2.6 Authentication and Security	xxii
2.7 Large Language Models and AI-Assisted Productivity	xxiv
2.8 Project Visualisation Paradigms	xxv
2.9 Summary	xxv
3 Methodology	xxvii
3.1 Requirements Analysis	xxvii
3.2 System Architecture Overview	xxix

3.3	Data Layer Design	xxx
3.4	Backend Implementation	xxxix
3.4.1	Authentication Service	xxxix
3.4.2	Task and Project Management	xxxix
3.4.3	Real-Time Notification System	xxxix
3.4.4	Messaging System	xxxix
3.4.5	Offline Synchronisation Engine	xxxix
3.4.6	AI Integration	xxxix
3.5	Frontend Implementation	xxxix
3.5.1	Application Structure	xxxix
3.5.2	Task Visualisation Views	xxxix
3.5.3	Connectivity Bar and Offline UX	xxxix
3.5.4	AI Chatbot Interface	xxxix
3.6	Desktop Packaging	xxxix
3.7	Testing Strategy	xxxix
4	Results	xxxix
4.1	System Features Overview	xxxix
4.2	Authentication and User Management	xl
4.3	Task and Project Management	xli
4.4	Task Visualisation Modes	xli
4.5	Team Collaboration and Messaging	xlii
4.6	Offline Synchronisation	xliii
4.7	AI Chatbot	xliv
4.8	Calendar and Dashboard	xlvi
4.9	Notifications and Settings	xlvi
4.10	Cross-Platform Distribution	xlvi
4.11	Performance Observations	xlvi
5	Conclusion	xlvi
5.1	Summary of Work	xlvi
5.2	Technical Contributions	xlvi
5.3	Lessons Learned	l
5.4	Limitations	li
6	Future Work	li
6.1	Short-Term Improvements	li
6.2	Long-Term Enhancements	lii
	Appendix	lv

A	API Reference	lvi
B	Database Schema	lviii
C	Configuration Reference	lx
	References	lxiv

List of Abbreviations

List of Figures

List of Tables

2.1	Feature comparison of popular task-management tools vs. Task Flow.	xix
3.1	MongoDB collections and their purpose.	xxxi
4.1	Functional requirement fulfilment matrix.	xl
4.2	Informal response-time observations for key operations.	xlvii
A.1	REST API controllers and primary endpoints.	lvii
B.1	SQLite entity summary.	lix
B.2	MongoDB collection descriptions.	lx
C.1	<code>appsettings.json</code> section reference.	lxi
C.2	Supported environment variables.	lxii
C.3	Key <code>electron-builder.json</code> configuration.	lxiii

Chapter 1

Introduction

1.1 Background and Motivation

The way people organise their work has changed fundamentally over the last decade. The proliferation of distributed teams, freelance arrangements, and hybrid work models has driven enormous growth in the market for digital task-management and project-collaboration software [1]. Tools such as Trello, Asana, Jira, and Notion have become everyday staples in software development teams, enabling members to track progress, share context, and coordinate across time zones without the overhead of physical meetings.

Despite this progress, the dominant generation of collaboration tools shares a critical architectural assumption: *a reliable, always-available internet connection*. Every create, read, update, and delete operation is routed through a remote cloud server. When connectivity degrades—as it does on mobile hotspots, in aeroplanes, in rural areas, or during network maintenance—these applications either display error banners or silently discard user input. For a professional whose productivity depends on continuous access to their task list, such fragility is unacceptable.

A second limitation is the growing gap between the capabilities of modern Large Language Models (LLMs) and the AI integrations available in mainstream productivity tools. Although several task managers have begun experimenting with AI-generated summaries, none provides a fully embedded, context-aware AI assistant that understands the user’s current project state and can assist with both natural-language planning and technical code generation within the same interface [2].

These two observations—the unreliability of purely cloud-centric architectures and the absence of deep AI integration—form the motivation for *Task Flow: a Smart Task & Quality Management System* designed from the ground up as an offline-first desktop application with an embedded multi-mode AI assistant.

1.2 Problem Statement

The central problem addressed by this project can be stated as follows:

Existing task-management tools treat internet connectivity as a prerequisite rather than an enhancement. They provide no graceful degradation path, no local data store, and no mechanism to synchronise divergent offline edits once connectivity is restored. Furthermore, their AI capabilities, where present, are superficial and disconnected from the user's live project data.

More concretely, a review of widely-used tools reveals the following gaps.

1. **No offline persistence.** Cloud-first tools store all state remotely. A user working without connectivity cannot create tasks, update statuses, or leave comments; changes attempted offline are either silently lost or surfaced as errors.
2. **No transparent synchronisation.** Even tools that offer limited offline caching typically give users no visibility into which changes are pending, in progress, or confirmed. The sync lifecycle is a black box, eroding trust in the accuracy of the displayed data.
3. **Fragmented toolchains.** Developers and project managers routinely switch between a task manager, a code assistant, a calendar, a messaging client, and a notification centre. There is no single, cohesive desktop application that integrates all of these capabilities into one consistent experience.
4. **Shallow AI integration.** AI features in existing tools are limited to simple text generation (e.g., auto-writing a task description). None provides a conversational assistant capable of code generation, document analysis, or multimodal input—all within the context of the user's own project data.

1.3 Aims and Objectives

The primary aim of this project is to design, implement, and evaluate a cross-platform desktop task-management application that operates reliably regardless of network availability, while embedding a powerful AI assistant and a full suite of collaboration features.

The following specific objectives guide the work:

1. **Offline-first operation.** Every core feature—task creation and editing, project management, calendar events, notifications, and messaging—must function fully without an active internet connection. Data must be persisted locally in a SQLite database that serves as the single source of truth for all user interactions.
2. **Reliable, transparent synchronisation.** When connectivity is available, locally generated changes must be propagated to a MongoDB relay database using a transactional

outbox pattern that guarantees at-least-once delivery. The user must be able to see the current connectivity status, the count of pending operations, and the real-time progress of any active sync pass through a dedicated UI indicator.

3. **Integrated multi-mode AI assistant.** A conversational AI assistant backed by the Mistral API must be embedded directly in the application. It must support three operational modes: a general-purpose chat mode, a code-generation mode powered by Codestral, and a document-analysis mode capable of processing uploaded images and PDF files via optical character recognition (OCR). Responses must be streamed to the UI via Server-Sent Events (SSE) to provide a responsive, real-time interaction feel.
4. **Five complementary task visualisation views.** The task module must offer five distinct views catering to different cognitive styles and workflows: a smart-grouped default list, a Kanban board with drag-and-drop, a tabular data-grid, a Gantt-style timeline, and a calendar time-grid.
5. **Cross-platform desktop distribution.** The completed application must be packaged as a native installer for Windows (NSIS), macOS (DMG), and Linux (AppImage/DEB) using Electron and electron-builder, bundling both the .NET backend and the React frontend into a single self-contained executable that requires no external runtime installation by the end user.
6. **Comprehensive collaboration feature set.** The system must include team management, one-to-one and group messaging with file attachments, real-time notifications via SignalR, calendar events, a dashboard with productivity statistics, and user-profile settings including security and preference controls.

1.4 Scope and Limitations

In Scope

The following capabilities were designed and implemented as part of this project:

- Full user authentication with JWT, bcrypt password hashing, and email-based password reset via MailKit.
- Task management with priority levels (Low, Medium, High), status workflow (To Do, In Progress, Review, Completed, Overdue), due-date tracking, starring, and assignment.
- Five task visualisation views: Default, Kanban, Table, Gantt, and Calendar.
- Project and team management with role-based membership and team invitations synchronised through MongoDB.

- One-to-one direct messaging and group chats, both supporting file attachments and soft-delete.
- In-application and email reminders triggered by configurable fire-at times.
- Automated due-date warnings fired 24 hours and 1 hour before deadline.
- Real-time push notifications delivered over SignalR.
- An AI chatbot with three modes, SSE streaming, and conversation history.
- Calendar events with a weekly time-grid view.
- A dashboard with live statistics, activity feeds, and a calendar widget.
- Offline-first SQLite persistence and an outbox-based MongoDB synchronisation subsystem.
- Cross-platform desktop packaging using Electron.

Out of Scope

The following items were explicitly excluded from the current implementation:

- **Mobile clients.** iOS and Android applications were not developed; the system targets desktop platforms only.
- **Multi-user conflict resolution.** The synchronisation subsystem uses a last-write-wins strategy. Collaborative real-time editing with Conflict-free Replicated Data Types (CRDTs) [3] is a known future enhancement.
- **Hierarchical sub-tasks.** A sub-task UI scaffold is present in the Table view but the backend persistence layer for sub-tasks was not implemented in this iteration.
- **SaaS deployment and billing.** The application is designed for single-user or small-team use on a local machine; multi-tenant cloud hosting and subscription management are out of scope.
- **Externally managed AI credentials.** The Mistral API key is currently supplied at build time. Secure external key management (e.g., system keychain or environment-variable injection) is identified as a known improvement area in Chapter 5.

1.5 Thesis Overview

The remainder of this thesis is structured as follows.

Chapter 2 — Literature Review and Background surveys the academic and technical context of the work, covering task-management paradigms, offline-first architecture, hybrid database design, real-time communication protocols, authentication, and the state of LLM-based AI assistants.

Chapter 3 — Methodology and System Design presents the full system architecture, the rationale behind key design decisions, the data model, and a detailed walkthrough of the back-end, frontend, and desktop-packaging implementations.

Chapter 4 — Results and System Evaluation demonstrates the implemented features through functional descriptions and user-interface walkthroughs, assessing the system against each objective stated in Section 1.3.

Chapter 5 — Conclusion reflects on the outcomes of the project, summarises its technical contributions, and acknowledges known limitations.

Chapter 6 — Future Work outlines a prioritised roadmap of short-term improvements and long-term enhancements that could extend the system.

Appendices provide supplementary technical reference material including the REST API endpoint catalogue, the complete database schema, and the application configuration reference.

Chapter 2

Literature Review

This chapter surveys the academic literature and technical background that underpins the design decisions made in Task Flow. The review spans seven areas: existing task-management tools and their limitations, desktop application frameworks, offline-first architecture patterns, hybrid database designs, real-time communication protocols, authentication and security, and the evolution of Large Language Models (LLMs) as productivity aids. It concludes with a brief treatment of project-visualisation paradigms.

2.1 Task Management and Productivity Software

Task management software encompasses a broad spectrum of tools—from simple personal to-do lists to enterprise project-management suites. The most widely cited personal productivity framework is David Allen’s *Getting Things Done* (GTD) [1], which argues that externalising tasks into a trusted system frees cognitive resources for execution rather than recall. Modern software tools attempt to operationalise this principle in varying ways.

Trello implements the Kanban paradigm [4] using cards on boards with user-defined columns. Its simplicity has made it popular for personal and small-team use, but it provides no offline capability; every drag-and-drop operation requires an active connection. **Asana** and **Jira** extend the model with structured project hierarchies, sprint planning, and reporting dashboards, but both are unambiguously cloud-first and offer only read-only offline caching at best. **Notion** introduces a flexible block-based editor that unifies notes, databases, and tasks in one surface, yet it shares the same connectivity dependency.

Table 2.1 summarises the key feature dimensions of these tools relative to Task Flow.

Table 2.1: Feature comparison of popular task-management tools vs. Task Flow.

Feature	Trello	Asana	Jira	Notion	Task Flow
Offline operation	×	×	×	×	✓
Local persistence	×	×	×	Limited	✓
Transparent sync	×	×	×	×	✓
Embedded AI assistant	Limited	Limited	Limited	Limited	✓
Desktop installer	×	×	×	✓	✓
Kanban view	✓	✓	✓	✓	✓
Gantt / timeline view	×	✓	✓	×	✓
Calendar view	×	✓	✓	✓	✓
Integrated messaging	×	×	×	×	✓

The pattern that emerges from this survey is consistent: each tool excels in one or two dimensions but requires multiple third-party integrations to achieve a comparable feature breadth to Task Flow, and none treats offline reliability as a design priority.

2.2 Desktop Application Frameworks

The choice of desktop application framework has long-term implications for developer productivity, binary size, platform reach, and the ability to reuse existing web technology investments.

Native frameworks such as WinForms, WPF (Windows), Cocoa (macOS), and GTK (Linux) offer tight OS integration and the best runtime performance but impose a per-platform development effort and cannot reuse the React/TypeScript frontend code already written for Task Flow.

Tauri [5] is a newer alternative to Electron that renders web content in the system’s native WebView and uses Rust for the application shell, resulting in significantly smaller binaries. However, its Rust back-end integration would require a full rewrite of the .NET server component, and its ecosystem maturity is substantially lower than Electron’s.

Electron [5], developed by GitHub and now maintained by the OpenJS Foundation, embeds a Chromium browser and a Node.js runtime in a single cross-platform binary. The web content layer communicates with the Node.js main process via inter-process communication (IPC). This architecture allows the complete React + TypeScript frontend to be reused without modification. For Task Flow, the .NET backend is launched as a child process by the Electron main process, with port negotiation performed via a token on standard output (`TASKFLOW_BACKEND_READY:{address}`). The resulting installer sizes are larger than Tauri equivalents but remain well within acceptable bounds for a desktop application. Electron’s comprehensive documentation, wide community adoption, and first-class support in

`electron-builder` [5] made it the optimal choice for this project.

2.3 Offline-First Architecture

The term *offline-first* was popularised in the context of progressive web applications and mobile clients, but the underlying principle applies equally to desktop software: *design the system to function fully without network access, and treat synchronisation as an enhancement rather than a prerequisite.*

Kleppmann’s *Designing Data-Intensive Applications* [6] provides a thorough treatment of the trade-offs involved. In the context of the CAP theorem [7], an offline-first system prioritises *Availability* and *Partition Tolerance* over strict *Consistency*: local writes are always accepted, and consistency is restored asynchronously when connectivity returns.

The Transactional Outbox Pattern

A key challenge in offline-first systems is ensuring that locally committed changes are eventually propagated to the remote store without duplication or loss. The *transactional outbox pattern* [8] addresses this by writing both the business entity change and a corresponding outbox entry in the same local database transaction. A background worker polls the outbox and replays pending entries to the remote system, advancing each entry through a state machine: Pending → Processing → Synced (or Failed after exhausting retries). This design guarantees at-least-once delivery while keeping the application logic decoupled from network availability.

Conflict Resolution

When multiple clients modify the same entity while offline, the system must resolve divergent versions upon reconnection. Conflict-free Replicated Data Types (CRDTs) [3, 9] provide a mathematically principled approach: data structures are designed so that any two states can be merged without coordination. Implementing CRDTs in a production system adds significant complexity; the current Task Flow implementation adopts a simpler *last-write-wins* strategy (using the `UpdatedAt` timestamp as the arbitration key), with CRDTs noted as a long-term enhancement.

2.4 Hybrid Database Architectures

SQLite as a Local Store

SQLite [10] is a self-contained, serverless, zero-configuration, transactional SQL database engine embedded as a library in the host application. It writes to a single file on disk, requires no background daemon process, and supports full ACID transactions. These properties make it the natural choice for a local persistence layer: the database travels with the application, survives process restarts, and performs all reads and writes without network round-trips. At the time of writing, SQLite is the most widely deployed database engine in the world, present in mobile operating systems, browsers, and billions of embedded devices.

In Task Flow, SQLite is the *single source of truth*. All application features read from and write to SQLite first; MongoDB involvement is entirely asynchronous and does not block the user-visible operation path.

MongoDB as a Cloud Relay

MongoDB [11] is a document-oriented database that stores data as BSON documents grouped in collections. Its flexible schema accommodates evolving data shapes without migration overhead and its horizontal scaling capabilities make it suitable for cloud deployment.

In Task Flow, MongoDB serves as a *synchronisation relay* for the subset of data that must be visible across devices or shared among team members: mirrored tasks, notifications, team invitations, and user-presence indicators. The document `_id` for each mirrored document is the entity's `SyncId` (a UUID generated at creation time), ensuring idempotent upserts and stable cross-device identity. This dual-database pattern—ACID-safe local SQLite as primary, schemaless MongoDB as asynchronous relay—is a pragmatic alternative to full CRDT-based synchronisation when conflict probability is low and eventual consistency is acceptable.

Entity Framework Core

Entity Framework Core [12] is the official Object-Relational Mapper (ORM) for .NET. It provides a LINQ-based query API, change tracking, and migration tooling that translates strongly typed C# entity classes into SQL DDL and DML statements. Its provider model allows the same application code to target SQLite in development and testing and PostgreSQL or SQL Server in production deployment without code changes—a property exploited via the `DatabaseProviderConfig` switch in Task Flow's `Startup.cs`.

2.5 Real-Time Communication Patterns

Modern collaborative applications require the server to *push* state changes to connected clients rather than waiting for clients to poll. Three primary mechanisms are relevant to Task Flow.

WebSockets

The WebSocket protocol [13] establishes a persistent, full-duplex TCP connection between a client and server after an HTTP upgrade handshake. Both parties may send frames at any time without the overhead of repeated HTTP headers. WebSockets are well-suited for low-latency bidirectional communication such as chat messages and live presence indicators.

SignalR

ASP.NET Core SignalR [14] is a higher-level abstraction over WebSockets, Server-Sent Events, and long polling. It provides a hub-based RPC model in which the server can invoke named methods on connected clients, and clients can invoke methods on the server. SignalR automatically negotiates the best available transport and handles reconnection. Task Flow uses SignalR's `NotificationHub` to broadcast real-time events: connectivity-state changes, synchronisation progress, task reminders, and new notification payloads are all delivered to the React client via a single hub connection.

Server-Sent Events

Server-Sent Events (SSE) is a unidirectional streaming protocol built on standard HTTP: the server holds the response connection open and writes `text/event-stream` chunks as data becomes available [14]. SSE is the appropriate choice for AI response streaming because the communication is inherently one-directional (server to client) and does not require the connection upgrade ceremony of WebSockets. Task Flow's `ChatbotController` exposes a `POST /api/chatbot/conversations/{id}/messages/stream` endpoint that writes Mistral API response chunks directly to the SSE response as they arrive, enabling the React frontend to render the AI reply token by token.

2.6 Authentication and Security

JSON Web Tokens

JSON Web Tokens (JWT) [15] are a compact, URL-safe means of representing claims between two parties. A JWT consists of three Base64URL-encoded sections—header, payload, and signature—concatenated with dots. The signature, computed using HMAC-SHA256 (HS256)

over the header and payload, allows the receiving party to verify the token’s integrity without querying a central session store. This stateless verification model is particularly valuable in a desktop application context where the backend is a local process: there is no shared session infrastructure between restarts. Task Flow issues JWTs with a 24-hour expiry; the signing key is injected at runtime via the `TASKFLOW_JWT_KEY` environment variable, falling back to a random 32-byte hex key generated at startup if none is provided.

Password Hashing with bcrypt

Storing passwords as plaintext or with reversible encryption is a critical security failure. The bcrypt algorithm [16] applies an adaptive key-derivation function with a configurable cost factor, deliberately making hash computation slow. As hardware performance improves, the cost factor can be increased to maintain the same brute-force resistance. Task Flow uses BCrypt.Net-Next with the default cost factor to hash passwords at registration and verify them at login, ensuring that even a full database compromise does not directly expose user credentials.

OWASP Top 10 Considerations

The Open Web Application Security Project (OWASP) publishes an annually updated list of the ten most critical web application security risks [17]. The most relevant to Task Flow are: *Broken Access Control*—mitigated by validating the JWT `sub` claim on every protected endpoint and checking entity ownership before returning or modifying data; *Cryptographic Failures*—addressed by bcrypt hashing and HS256 JWT signing; *Injection*—mitigated by EF Core’s parameterised queries which eliminate raw SQL string concatenation; and *Security Misconfiguration*—acknowledged in the context of the Mistral API key, which should be supplied via an environment variable rather than embedded in source code.

Role-Based Access Control

Ferraiolo and Kuhn’s foundational work on role-based access control (RBAC) [18] defines a model in which permissions are assigned to roles and roles are assigned to users, rather than assigning permissions directly to users. Task Flow implements a lightweight RBAC model for team membership: each `TeamMember` record carries a `Role` field, and the `Teams` controller enforces that only the team owner may perform administrative operations such as removing members or disbanding the team.

2.7 Large Language Models and AI-Assisted Productivity

The Transformer Architecture

The transformer architecture, introduced by Vaswani et al. [19] in the seminal paper *Attention Is All You Need*, replaced recurrent sequence-to-sequence models with a purely attention-based mechanism. The self-attention operation allows each token in a sequence to attend to every other token in parallel, enabling efficient training on large corpora and strong performance on a wide range of language tasks. This architecture became the foundation for all major LLMs in subsequent years.

GPT-3 and the Emergence of Instruction Following

Brown et al.’s GPT-3 [20] demonstrated that scaling transformer models to 175 billion parameters produced qualitatively new capabilities, including few-shot task completion from natural-language prompts. The subsequent development of instruction-tuned variants established the prompt-response interaction model that underpins modern conversational AI assistants.

Mistral Models

Jiang et al. [2] introduced Mistral 7B, a 7-billion-parameter open-weight model that outperformed LLaMA 2 13B on several benchmarks despite being half the size, achieved through grouped-query attention and sliding-window attention mechanisms. Task Flow integrates the Mistral API at three model tiers: `mistral-small-latest` for general conversational use; `codestral-latest` for the code-generation mode, a model fine-tuned on a large corpus of programming languages; and `mistral-ocr-latest` for the document-scanner mode, which accepts base64-encoded images and PDF files and returns extracted and structured text. The 20-message conversation history window provides conversational context while bounding the per-request token count.

AI-Assisted Productivity Research

Recent studies examining AI coding assistants report productivity gains of 26–55% on specific programming tasks, with larger gains on routine boilerplate generation and smaller gains on complex architectural reasoning. The integration of an AI assistant directly within the task-management interface—rather than requiring the user to switch to a separate tool—reduces context-switching overhead and allows the assistant to be consulted in the flow of work.

2.8 Project Visualisation Paradigms

Different users hold different mental models of their work, and no single visualisation paradigm suits all cognitive styles or project types. A mature task-management tool should offer multiple views over the same underlying data.

Kanban boards originated in Toyota’s production system as physical cards (*kanban* = “signal card” in Japanese) that controlled the flow of work between manufacturing stages [4]. Their digital equivalent presents tasks as cards in columns representing workflow stages, giving teams an immediate visual signal of bottlenecks (columns with too many cards) and idle capacity. Drag-and-drop interaction directly maps the physical act of moving a card from one column to the next.

Gantt charts were developed by Henry Gantt in the early twentieth century as project-schedule representations that map tasks onto a horizontal time axis [21]. They are particularly effective for deadline-sensitive planning because the chart immediately communicates which tasks are in progress, which are complete, and which are overdue relative to a calendar.

Calendar views represent tasks as events on a time grid, a mental model familiar to users from personal calendars. A weekly or monthly grid makes it easy to reason about task density and identify overloaded days.

Tabular views surface all task attributes as a structured grid, enabling rapid scanning, sorting, and filtering without the spatial metaphor of Kanban columns or timeline bars. This view is preferred by users who think in terms of data rather than process flow.

Task Flow provides all four of these paradigms plus a smart-grouped default list (grouping tasks by temporal proximity: Overdue, Today, This Week, Later, Completed), giving each user the freedom to work in the style that best fits their workflow at any given moment.

2.9 Summary

The literature review reveals a clear pattern of unmet needs in the current landscape of task-management software. Existing tools are built on cloud-first assumptions that render them fragile in low-connectivity environments. The offline-first literature, particularly the transactional outbox pattern described by Fowler [8] and the data-intensive application principles formalised by Kleppmann [6], provides the theoretical basis for a reliable local-first persistence and synchronisation subsystem. The dual-database architecture combining SQLite and MongoDB is a pragmatic application of these principles that avoids the complexity of full CRDT-based conflict resolution while still enabling cross-device data relay.

In the domain of AI integration, the Mistral family of models [2] offers an open-weight, commercially accessible set of capabilities—general language, code generation, and OCR—

that align precisely with the three modes required by the system. Server-Sent Events provide the appropriate transport mechanism for streaming AI responses to the client with minimal protocol overhead.

Electron's maturity and React-reuse properties make it the most pragmatic desktop packaging choice, while JWT and bcrypt together provide a well-understood and academically grounded authentication and security foundation.

The following chapter translates these theoretical foundations into a concrete system design, detailing the architecture, data model, and implementation of each subsystem in Task Flow.

Chapter 3

Methodology

This chapter describes the methodology used to design and implement Task Flow. It begins with a structured requirements analysis, then presents the high-level system architecture, followed by detailed accounts of the data layer, backend, frontend, and desktop-packaging implementations. The chapter closes with a discussion of the testing strategy applied throughout development.

3.1 Requirements Analysis

Functional Requirements

The following functional requirements were identified from the project objectives stated in Section 1.3:

1. **FR-01 User Authentication.** The system shall support user registration, login, and password reset via email. Passwords shall be stored as bcrypt hashes; sessions shall be managed with signed JWT tokens valid for 24 hours.
2. **FR-02 Task Management.** Users shall be able to create, read, update, and delete tasks. Each task shall carry a title, optional description, priority (Low, Medium, High), status (To Do, In Progress, Review, Completed, Overdue), optional due date, star flag, and project assignment.
3. **FR-03 Multiple Task Views.** Tasks shall be displayable in five views: a smart-grouped default list, a Kanban board, a tabular grid, a Gantt timeline, and a calendar time-grid.
4. **FR-04 Project and Team Management.** Users shall be able to create projects, invite team members, and assign tasks to projects. Team invitations shall be transmitted via MongoDB to support cross-device reception.

5. **FR-05 Messaging.** Users shall be able to exchange one-to-one direct messages and participate in group chats. Both conversation types shall support file attachments up to 50 MB.
6. **FR-06 Notifications and Reminders.** The system shall deliver real-time in-application notifications over SignalR and optional email reminders at user-configured times. Automated warnings shall fire 24 hours and 1 hour before task due dates.
7. **FR-07 Calendar Events.** Users shall be able to create, edit, and delete calendar events with start and end times, an optional colour, and an optional meeting link.
8. **FR-08 AI Chatbot.** A conversational AI assistant shall be available in three modes: general chat, code generation, and document scanning (OCR). Responses shall be streamed token-by-token via SSE. Conversation history shall be persisted between sessions.
9. **FR-09 Offline Operation.** All features in FR-01 through FR-07 shall function without network access. Locally generated changes shall be queued and replayed to MongoDB when connectivity is restored.
10. **FR-10 Dashboard.** A dashboard shall display summary statistics (active tasks, in-progress count, project count, team member count), a task-trend indicator, tasks due this week, a recent-activity feed, and an embedded calendar widget.
11. **FR-11 Settings.** Users shall be able to update their profile (name, avatar, company, country, phone, timezone), change their password, and manage notification preferences.

Non-Functional Requirements

1. **NFR-01 Offline Availability.** The application shall start and remain fully operable on a machine with no network access.
2. **NFR-02 Security.** All API endpoints shall require a valid JWT. Passwords shall not be stored in recoverable form. The application shall not expose SQL injection vectors.
3. **NFR-03 Cross-Platform Distribution.** The packaged installer shall run on Windows (x64), macOS (x64/ARM), and Linux (x64) without requiring the user to install any additional runtime.
4. **NFR-04 Responsiveness.** UI interactions shall produce visible feedback within 200 ms under normal conditions.
5. **NFR-05 Extensibility.** The backend shall use a repository and service pattern such that new entity types can be added without modifying existing controller logic.

3.2 System Architecture Overview

Task Flow is structured as a three-tier desktop application embedded in an Electron shell.

Tier 1 — Presentation (React + TypeScript). The user interface is a single-page application (SPA) built with React 19 and TypeScript 6. It communicates with the backend exclusively via HTTP REST calls and a persistent SignalR WebSocket connection. Webpack 5 bundles all TypeScript and Tailwind CSS sources into a set of static assets served from `wwwroot/` by the Kestrel HTTP server.

Tier 2 — Application Logic (ASP.NET Core 10). The backend exposes 14 REST API controllers grouped under `/api/`, one SignalR hub (`/hubs/notifications`), and four background services. It follows a layered architecture: controllers delegate to service classes, which in turn use repository objects (EF Core `DbContext` queries) for persistence. AutoMapper projects entity objects onto DTOs at the service boundary, preventing internal model details from leaking into the API response. A global `ExceptionHandlerMiddleware` catches unhandled exceptions and returns a uniform JSON error envelope, ensuring no stack trace is ever exposed to the client.

Tier 3 — Data (SQLite + MongoDB). SQLite serves as the primary, always-available local store. MongoDB is an optional cloud relay used for cross-device features. The application starts and runs normally even when the MongoDB connection string is empty or the server is unreachable.

Electron Shell. The Electron main process (`public/electron.js`) launches the compiled .NET backend as a child process. It monitors the child's standard output stream and waits for the line `TASKFLOW_BACKEND_READY:{address}` before opening the browser window and pointing it at the dynamically assigned `http://127.0.0.1:{port}` address. This design ensures the UI is never shown before the API is ready to serve requests.

Request Lifecycle. A typical authenticated request follows the path: React component → `fetch()` with `Authorization: Bearer {token}` header → Kestrel → JWT middleware validates signature and extracts claims → controller action → service → EF Core → SQLite. If the operation mutates a syncable entity, the service additionally calls `IMirrorService.Mirror` which either performs a direct MongoDB upsert (online) or enqueues a `SyncOutboxEntry` (offline). Real-time events are broadcast to the connected React client via the SignalR hub independently of the HTTP response path.

3.3 Data Layer Design

The ISyncableEntity Interface

Entities that must be mirrored to MongoDB implement the `ISyncableEntity` interface, which exposes three properties:

- `SyncId` (GUID) — a stable, globally unique identifier assigned at creation time. It is used as the MongoDB document `_id`, enabling idempotent upserts: re-running the same outbox entry twice produces the same document, not a duplicate.
- `UpdatedAt` (DateTime) — the wall-clock timestamp of the most recent change, used as the arbitration key in last-write-wins conflict resolution.
- `IsSynced` (bool) — a dirty flag that indicates whether the local record has been propagated to MongoDB.

The `AppDbContext.SaveChangesAsync()` override automatically stamps `UpdatedAt` to `DateTime.UtcNow` and assigns a new `SyncId` for newly added entities, eliminating boilerplate from every service call. It also sets `IsSynced = false` for every modified entity to ensure that outstanding changes are detected by the sync engine.

SQLite Entity Model

The application manages eighteen entity types in SQLite, grouped as follows:

- **Identity:** `AppUser`
- **Work:** `TaskItem`, `TaskComment`, `Project`, `ProjectMember`
- **Team & Social:** `Team`, `TeamMember`, `LocalTeamMember`, `LocalInvitation`
- **Messaging:** `Message`, `GroupChat`, `GroupChatMember`, `GroupMessage`
- **Time:** `CalendarEvent`, `Reminder`
- **Alerts:** `Notification`
- **AI:** `ChatbotConversation`, `ChatbotMessage`
- **Sync:** `SyncOutboxEntry`

Unique indexes on `SyncId` for the four core syncable types (`Project`, `TaskItem`, `Notification`, `Reminder`) prevent duplicate inserts if the application is restarted mid-sync. `LocalInvitation` and `LocalTeamMember` are read-only mirrors of MongoDB documents pulled on login; their `MongoId` field serves as the local primary key.

MongoDB Collection Design

Five collections are maintained in the `TaskFlow` MongoDB database:

Table 3.1: MongoDB collections and their purpose.

Collection	Key Index	Purpose
tasks	<code>_id</code> (SyncId)	Mirror of SQLite task rows
notifications	<code>_id</code> (SyncId)	Mirror of SQLite notifications
team_invitations	(recipientEmail, status)	Cross-device team invites
team_members	(teamId, ownerEmail)	Team membership roster
user_presence	email (unique)	Online/offline status

3.4 Backend Implementation

3.4.1 Authentication Service

User registration follows a three-step process: (1) validate the input DTO using `FluentValidation` [22]; (2) hash the password with `BCrypt.Net-Next` using the default cost factor; (3) persist the `AppUser` record to SQLite and asynchronously mirror a safe projection of the user to MongoDB, enabling other team members to look up the user by email when sending invitations.

Login validates the submitted password against the stored hash using `BCrypt.Verify()`. On success, the service constructs a JWT containing ten claims (`id`, `email`, `given_name`, `family_name`, `avatarUrl`, `company`, `country`, `phone`, `timezone`, and `jti`). The token is signed with HMAC-SHA256 using the key resolved from the `TASKFLOW_JWT_KEY` environment variable (falling back to the `Jwt:Key` configuration value, or an auto-generated 32-byte random hex string if neither is present). A fire-and-forget call to `UserDataSyncService.PullFor` pulls the latest MongoDB data for that user immediately after login, refreshing local copies of team memberships and pending invitations.

Password reset is implemented via a time-limited token: a random URL-safe token is stored in `AppUser.ResetToken` alongside a `ResetTokenExpiry` timestamp and emailed to the user using MailKit over Gmail SMTP (port 587, STARTTLS). The reset endpoint validates the token, checks expiry, and updates the password hash.

3.4.2 Task and Project Management

`TaskItem` is the central entity of the system. Each task belongs to exactly one `AppUser` (its assignee) and optionally to one `Project`. The `Priority` enum defines three levels: Low (0), Medium (1), and High (2). The `TaskItemStatus` enum defines five states: `Todo`,

InProgress, Review, Completed, and Overdue. The Overdue state is not set manually; it is assigned automatically by the `DueDateWarningService` background service when the due date passes.

All `TaskService` operations validate ownership by comparing the `AssigneeId` of the fetched entity against the `userId` extracted from the JWT claim `ClaimTypes.NameIdentifier`, returning a 403 response for unauthorised access attempts. Text search is executed at the database layer using EF Core `.Where()` with `string.Contains()` predicates (compiled to SQL `LIKE` clauses), avoiding in-memory filtering of large datasets. AutoMapper projects `TaskItem` entities onto `TaskItemDto` response objects, exposing only the fields needed by the frontend.

After any create or update operation, the service invokes `IMirrorService.MirrorAsync(entity, operationName)`. `MirrorService` checks `ConnectivityService.IsEffectivelyOnline`: if online, it calls `MongoService.UpsertDocumentBySyncIdAsync()` directly; if offline, it serialises the entity payload to JSON and inserts a `SyncOutboxEntry` into the local database with status `Pending`.

3.4.3 Real-Time Notification System

`NotificationHub` extends ASP.NET Core SignalR's `Hub` class. On connection, each client is added to a user-specific SignalR group named `user_{userId}`. The hub reads the JWT from the `access_token` query string parameter (required because SignalR's JavaScript client cannot set HTTP headers during the WebSocket upgrade). The unread notification count is pushed to the client immediately upon connection.

The system defines fifteen notification types (e.g., `TaskAssigned`, `TaskCompleted`, `MessageReceived`, `TeamInvitation`, `TaskOverdue`) and four priority levels (`Low`, `Normal`, `High`, `Urgent`). Each notification includes an `ActionUrl` field that the frontend uses to navigate the user to the relevant resource on click.

Two background services supplement the notification hub: `DueDateWarningService` runs on a 1-minute timer, queries all tasks with approaching due dates, and fires both in-app notifications and (if configured) email alerts. An in-memory `_sentNotifications` dictionary deduplicates warnings so that the same task does not generate repeated notifications within the same application session. `ReminderProcessorService` runs on a 1-minute timer and fires user-configured `Reminder` records whose `FireAt` time has passed and whose `HasFired` flag is still false.

3.4.4 Messaging System

The messaging module provides two conversation types.

Direct Messages. The `Message` entity stores a sender, a receiver, a text body, a read flag, and separate soft-delete flags for each party (`IsDeletedBySender`, `IsDeletedByReceiver`). This design ensures that a message deleted by one party remains visible to the other until they also delete it. File attachments (URL, original file name, MIME type, size in bytes) are stored as nullable fields on the `Message` entity and are limited to 50 MB by Kestrel's request body size limit. The `MessagesController` sends a `MessageReceived` SignalR event to the recipient's user group after persisting each message, enabling real-time delivery without polling.

Group Chats. `GroupChat` stores a name, description, and owner. `GroupChatMember` is the join entity between a group and its participants. `GroupMessage` mirrors the attachment model of direct messages and additionally records whether the message has been edited (`IsEdited`). The `GroupChatsController` broadcasts a `GroupMessageReceived` event to a SignalR group named after the group chat ID.

3.4.5 Offline Synchronisation Engine

The synchronisation subsystem is composed of three cooperating components.

ConnectivityService is a singleton that maintains the effective connectivity state of the application. It exposes `IsOnline` (result of the last MongoDB ping), `IsManualOffline` (user-toggled flag from the UI), and the derived `IsEffectivelyOnline` property (`IsOnline && !IsManualOffline`). It also exposes `PendingSyncCount` (count of `SyncOutboxEntry` rows with status `Pending` or `Failed`) and a `ConnectivityChanged` event that subscribers can observe.

OfflineSyncService is the background worker that replays the outbox. It runs a 10-second ping loop: on each tick it calls `MongoService.PingAsync()` and updates `ConnectivityService`. It also subscribes to `ConnectivityChanged`: when the state transitions from offline to online, it waits 1500 ms (allowing the MongoDB connection to stabilise) and then calls `ReplayOutboxAsync`. During replay, each `Pending` or `Failed` outbox entry is advanced to `Processing`, the stored operation is executed against MongoDB, and the entry is marked `Synced` on success or `Failed` (with an incremented attempt counter and error message) on failure. SignalR events (`SyncStarted`, `SyncProgress`, `SyncComplete`) are broadcast to the client throughout the replay process, feeding the `ConnectivityBar`'s animated progress indicator.

BulkSyncStartupService runs once at application startup. It waits up to 90 seconds for MongoDB to become reachable, then performs a bulk upsert of all SQLite entities implementing `ISyncableEntity`. This ensures that a fresh installation, or a machine that was offline during the previous session, fully reconciles its local state with MongoDB before accepting new writes.

3.4.6 AI Integration

`MistralChatService` encapsulates all communication with the Mistral REST API. It is constructed as a typed `HttpClient` singleton registered in the DI container. Three operational modes are supported, each targeting a different model and system prompt:

- **General mode** (`mistral-small-latest`) — a general-purpose assistant that can answer questions, help with planning, and discuss any topic.
- **Coder mode** (`codestral-latest`) — a code-specialised assistant with a system prompt that instructs the model to prefer code examples and to explain algorithmic choices.
- **Scanner mode** (`mistral-ocr-latest`) — a vision-language mode that accepts base64-encoded images or PDF documents and returns extracted, structured text. PDFs are submitted to the `/v1/ocr` endpoint; images are embedded directly in the chat message payload.

Streaming. The `ChatAsync()` method makes an HTTP POST request with `stream: true` in the request body and reads the response using `HttpCompletionOption.ResponseHeadersRead`, then processes the response body as an `IAsyncEnumerable<string>` of SSE lines. Each line prefixed with `data:` is deserialised as a chat-completion delta, and the token content is written immediately to the ASP.NET Core SSE response stream using `Response.WriteAsync()`. This pipeline adds negligible latency between the Mistral API producing a token and the React frontend rendering it.

History window. The service loads the last 20 messages from the `ChatbotMessage` table before constructing the API request, providing the model with sufficient context while bounding the token cost of each request. After streaming completes, the assistant's full response is assembled from all received deltas and persisted as a new `ChatbotMessage` row with `Role = "assistant"`.

Title generation. When a new conversation is started, a follow-up call to `GenerateTitleAsync()` asks the model to produce a short, descriptive title from the first user message. This title is stored in `ChatbotConversation.Title` and displayed in the conversation list sidebar.

3.5 Frontend Implementation

3.5.1 Application Structure

The React application is organised as a flat module tree rooted at `ReactApp/`. React Router 7 manages client-side navigation with a route table of seventeen pages. All routes except `/login`, `/signup`, `/forgot-password`, and `/reset-password` are wrapped in an `<AuthGuard>`

component that redirects unauthenticated users to the login page.

Styling is managed entirely with Tailwind CSS 4 utility classes; no custom CSS files exist in the project. The Lucide React icon library provides consistent SVG icons throughout the UI. Radix UI primitives (`@radix-ui/react-dropdown-menu`) are used for accessible drop-down menus. The `react-markdown` library with the `remark-gfm` plugin renders assistant responses as formatted Markdown, supporting code blocks, tables, and inline formatting.

Webpack 5 transpiles TypeScript via `ts-loader` and PostCSS processes Tailwind classes; the output bundles are written to `wwwroot/dist/` and served directly by Kestrel.

3.5.2 Task Visualisation Views

All five views consume the same `TaskItem[]` array from a shared context and differ only in their rendering and interaction model.

Default View. Tasks are partitioned into five temporal groups computed client-side: Overdue (past due date), Today (due today), This Week (due within 7 days), Later (due after 7 days), and Completed. Each group is rendered as a collapsible section with a task count badge. This grouping replicates the mental model of a physical in-tray sorted by urgency.

Kanban View. Five columns correspond to the five task statuses. Each column header is coloured to convey urgency (red for Overdue, amber for In Progress, etc.) and displays the column's task count. Tasks are rendered as drag-and-drop cards using the HTML5 Drag and Drop API; dropping a card in a different column fires a `PATCH /api/tasks/{id}` request to update the task's status.

Table View. Tasks are displayed in a columnar grid with columns for title, status, priority, due date, project, and actions. Status and priority are rendered as colour-coded badge components. Each row has an expand toggle that reveals a sub-task scaffold row (UI-level only; sub-task backend persistence is a planned future feature).

Gantt View. A full-calendar-year timeline is rendered as a horizontal structure. Each task is represented as a bar positioned and sized according to its due date. A priority colour legend (red = High, amber = Medium, green = Low) allows at-a-glance workload assessment. The current date is highlighted with a vertical marker.

Calendar View. A weekly time-grid spanning 7:00 AM to 8:00 PM displays tasks and calendar events as positioned blocks. A dark mini-calendar sidebar allows week navigation. Tasks without a specific time are shown as all-day entries at the top of the grid.

3.5.3 Connectivity Bar and Offline UX

`ConnectivityBar.tsx` is a fixed-position banner rendered at the top of the application layout. It subscribes to real-time SignalR events (`ConnectivityChanged`, `SyncStarted`, `SyncProgress`, `SyncComplete`) via the `useConnectivity()` custom hook.

When the application is offline, the banner shows in amber with an offline icon and the count of pending sync operations. When a sync pass is in progress, the banner shows in blue and renders an animated progress bar that advances as each outbox entry is processed. When connectivity is healthy and no sync is in progress, the banner is hidden entirely to avoid distracting the user. A `toggleManualOffline` button allows developers and testers to simulate offline mode without physically disconnecting the network.

3.5.4 AI Chatbot Interface

`Chatbot.tsx` is the most complex single component in the application. Three toggle buttons at the top of the chat panel select the active mode (General, Coder, Scanner); the active mode determines which API model is used on the backend.

When no conversation is active, a grid of starter-prompt buttons provides suggested questions appropriate for each mode, helping new users understand the assistant's capabilities.

Messages are submitted via a form that supports both text and a single file attachment (triggered by a paperclip icon). The component opens an `EventSource` connection to the SSE endpoint for each new message and appends incoming token chunks to the current assistant message bubble as they arrive. When the SSE stream closes, the final assembled message is rendered through `react-markdown` with syntax-highlighted code blocks.

A collapsible sidebar lists all saved conversations, sorted by most-recently-updated. Each conversation displays its AI-generated title and a relative timestamp. Clicking a conversation loads its full message history from the backend.

3.6 Desktop Packaging

The production build of Task Flow is a self-contained desktop installer that bundles three components: the Electron shell, the compiled .NET backend, and the Webpack-built React frontend assets.

Electron main process. `public/electron.js` is the entry point for the desktop application. On startup it spawns the compiled .NET backend executable as a child process, capturing its standard output. The main process watches for the line matching `TASKFLOW_BACKEND_READY: {add` to appear. Once detected, it extracts the port number, constructs the backend URL, and calls

`mainWindow.loadURL()` to load the React SPA from the local Kestrel server. If the back-end process exits unexpectedly, the main process terminates the Electron application with an error dialog.

CORS configuration. Because Electron loads the frontend from a `file://` origin (for the production build) or from `localhost` (during development), the ASP.NET Core CORS policy uses `SetIsOriginAllowed(_ => true)` combined with `AllowCredentials()` to permit the SignalR connection from either origin without hard-coding specific URLs.

JWT key management. The Kestrel server listens on `http://127.0.0.1:0` (a random available port). The JWT signing key is resolved in order of priority: (1) `TASKFLOW_JWT_KEY` environment variable; (2) `Jwt:Key` value in `appsettings.json`; (3) an automatically generated 32-byte random hex string persisted for the duration of the process. This means the application works out-of-the-box without any configuration while still allowing a stable key to be injected for production deployments.

electron-builder configuration. `electron-builder.json` configures the following target artefacts:

- **Windows** — NSIS installer with `oneClick: false`, allowing the user to choose the installation directory and optionally create desktop and Start Menu shortcuts.
- **macOS** — DMG image and ZIP archive (for notarisation-free distribution).
- **Linux** — AppImage (portable, no installation required) and DEB package (for Debian/Ubuntu-based distributions).

The application ID is `com.taskflow.desktop`. The .NET backend self-contained publish output (`win-x64`, `osx-x64`, `linux-x64`) is included as an extra resource so that electron-builder packages the correct native binary for each target platform.

3.7 Testing Strategy

Testing was conducted at three levels throughout the development process.

Unit-level verification. Service-layer methods were verified by constructing the service with an in-memory SQLite database (using EF Core's `UseInMemoryDatabase` provider) and asserting the expected state after each operation. JWT claim extraction and bcrypt hash/verify round-trips were tested in isolation.

API integration testing. Each of the 14 controllers was exercised against a running Kestrel instance using `HttpClient` requests constructed with valid and invalid JWT tokens. The tests verified: (a) that protected endpoints return 401 when no token is supplied; (b) that ownership checks return 403 for resources belonging to a different user; and (c) that create, read,

update, and delete operations produce the expected HTTP status codes and response bodies.

Offline synchronisation testing. The offline sync pipeline was validated using the `ConnectivityBar`'s manual offline toggle. With `IsManualOffline = true`, a series of task create and update operations was performed; the outbox was inspected to confirm that `SyncOutboxEntry` rows were created with status `Pending`. On toggling back online, the sync replay was observed via the progress bar and the MongoDB collection was inspected to confirm all entries were promoted to `Synced`.

Cross-platform packaging. The NSIS installer was tested on Windows 11 and Windows 10. The macOS DMG was verified on macOS 14 (Sonoma) with both Intel and Apple Silicon (via Rosetta 2 emulation). The AppImage was tested on Ubuntu 22.04. In all cases the application launched, authenticated, and performed offline task creation and synchronisation without errors.

Chapter 4

Results

This chapter presents the functional outcomes of the Task Flow implementation. Each section evaluates a major feature area against the requirements established in Chapter 3, describes the implemented behaviour, and notes any deviations or limitations observed during development and testing.

4.1 System Features Overview

Table 4.1 maps each functional requirement from Section 3.1 to its implementation status in the delivered system.

Table 4.1: Functional requirement fulfilment matrix.

Req.	Status	Notes
FR-01 User Authentication	Fully met	Registration, login, JWT, password reset via email
FR-02 Task Management	Fully met	Full CRUD, all status/priority values, star flag
FR-03 Multiple Task Views	Fully met	Default, Kanban, Table, Gantt, Calendar
FR-04 Project & Teams	Fully met	Projects, membership, MongoDB-relayed invitations
FR-05 Messaging	Fully met	DMs, group chats, file attachments up to 50 MB
FR-06 Notifications	Fully met	SignalR, email, 24 h and 1 h due-date warnings
FR-07 Calendar Events	Fully met	Full CRUD, colour, meeting link, weekly time-grid
FR-08 AI Chatbot	Fully met	Three modes, SSE streaming, persisted history
FR-09 Offline Operation	Fully met	Outbox queue, auto-replay, BulkSync on startup
FR-10 Dashboard	Fully met	Stats, trend, due-this-week, recent activity
FR-11 Settings	Fully met	Profile, password change, notification preferences

All eleven functional requirements and all five non-functional requirements were met in the final build. The one outstanding item noted in the non-functional requirements is NFR-02 (Security): the Mistral API key is currently stored as a hard-coded constant inside `MistralChatService.cs` rather than being injected via an environment variable or secrets manager. This is documented as a known limitation and discussed further in Chapter 5.

4.2 Authentication and User Management

The authentication flow was implemented in full. New users complete a registration form requiring first name, last name, email address, and password. `FluentValidation` rejects passwords shorter than eight characters, empty names, and malformed email addresses before the request reaches the service layer. Passwords are stored as `BCrypt` hashes (default cost factor 11); plain-text passwords are never written to the database or log files.

Upon successful login the API returns a signed JWT containing ten claims. The React client

stores this token in `localStorage` and attaches it to every subsequent API request via the `Authorization: Bearer` header. An `<AuthGuard>` component checks token presence on each route change and redirects unauthenticated users to the login page immediately, without a round-trip to the server.

The password reset flow was verified end-to-end: the `/api/auth/forgot-password` endpoint generates a URL-safe random token, stores it with a 1-hour expiry in `AppUser.ResetToken`, and dispatches a reset-link email via `MailKit`. The reset endpoint validates the token and expiry, applies the new `BCrypt` hash, and clears the token fields to prevent re-use.

User profile data (avatar URL, company, country, phone, timezone) is embedded in the JWT at login, making it available to the React application without an extra API call. Profile updates are reflected in the next token refresh (next login).

4.3 Task and Project Management

Task management is the core of the application. The `TasksController` exposes nine endpoints covering create, read (list and single), update (full and partial status-only `PATCH`), delete, toggle-star, search, and bulk-action operations. All endpoints enforce ownership: a user can only read, edit, or delete tasks where `AssigneeId` matches their JWT identity. Attempts to access another user's tasks return HTTP 403.

Tasks may be assigned to one of three priority levels and five status values. The `DueDateWarningService` automatically transitions tasks whose due date has passed into the `Overdue` status on its next 1-minute polling cycle, ensuring the status always reflects reality without requiring user action.

Projects are created with a name, optional description, and optional colour. The project owner can invite collaborators by email; the invitation is published to the `team_invitations` MongoDB collection and retrieved by the invitee on their next login. Accepting an invitation creates a `ProjectMember` row in the invitee's local SQLite database. Task assignment within a project is constrained to project members.

Task comments were implemented as a standalone entity linked to `TaskItem`. Each comment stores the author, body text, and creation timestamp. Creating a comment triggers a `TaskCommented` `SignalR` notification to the task owner (if different from the commenter).

4.4 Task Visualisation Modes

Five distinct visualisation modes were implemented and verified.

Default View. Tasks are partitioned client-side into five temporal groups: *Overdue*, *Today*, *This Week*, *Later*, and *Completed*. Each group renders as a collapsible section with a count

badge. An inline quick-create form at the top of each section allows tasks to be added without opening the full task editor. This view proved most useful for daily triage.

Kanban View. Five columns represent the five task statuses. Cards are moved between columns via the HTML5 Drag and Drop API; on drop, a `PATCH` request updates the task's status and the outbox entry is enqueued if offline. Column headers display per-column task counts and are colour-coded: red for *Overdue*, amber for *In Progress*, blue for *Review*, green for *Completed*, and grey for *To Do*. The drag affordance was verified to work on Windows 11 (Chrome and Electron) and macOS 14 (Safari and Electron).

Table View. Tasks are displayed in a data-grid layout with sortable columns for title, status, priority, due date, and project. Status and priority cells render as colour-coded pill badges. Each row includes an expand control that reveals a placeholder sub-task row; actual sub-task persistence has been deferred to a future release (see Chapter 6).

Gantt View. A full-calendar-year timeline renders task bars positioned by due date. Tasks without a due date are excluded from this view. A priority legend (red = High, amber = Medium, green = Low) is rendered below the timeline header. A vertical rule marks the current date. During testing the Gantt view was found useful for identifying workload clusters across months.

Calendar View. A weekly time-grid covering 7:00 AM to 8:00 PM displays both `TaskItem` records and `CalendarEvent` records as positioned blocks. A dark-themed mini-calendar in the left sidebar allows week navigation. Tasks without a time component appear as all-day entries in a banner row at the top of the grid. The grid correctly handles events that overlap in time by rendering them in adjacent lanes.

4.5 Team Collaboration and Messaging

Team and Project Invitations

The team invitation workflow was tested end-to-end across two separate application instances running on the same machine under different user accounts. The inviting user entered the recipient's email address; the invitation document was written to MongoDB. On the recipient's next login, `UserDataSyncService.PullForUserAsync()` fetched the pending invitation and inserted a `LocalInvitation` row into the recipient's SQLite database. The invitation appeared in the recipient's notification list; accepting it created the `TeamMember` or `ProjectMember` record as expected.

Direct Messaging

The direct messaging system supports full-duplex real-time text exchange. The `MessagesController` persists each message and immediately broadcasts a `MessageReceived` SignalR event to the recipient's user group. On the receiving side, the React client increments the unread badge on the `Messages` navigation icon without requiring a page refresh. Soft-delete was verified: after one party deleted a conversation, their message list was empty while the other party's list remained intact. File attachments were tested with a 2 MB PNG image and a 15 MB PDF; both uploaded successfully and the attachment metadata was stored on the `Message` entity.

Group Chats

Group chats support up to an arbitrary number of participants. The group owner can add members after creation; each new member receives a `GroupInvitation` notification. Messages sent to a group are broadcast to a SignalR group named after the `GroupChat.Id`, ensuring that all connected participants receive the message simultaneously. Message editing (`IsEdited` flag) and attachment support were both verified.

4.6 Offline Synchronisation

The offline synchronisation subsystem was the most technically complex feature of the project and received the most thorough testing.

Outbox Queue Behaviour

With the `ConnectivityBar`'s manual offline toggle activated (`IsManualOffline = true`), twenty task create and update operations were performed sequentially. Inspection of the SQLite database confirmed that twenty `SyncOutboxEntry` rows were created, each with status `Pending`, a serialised JSON payload, and the correct operation name (`UpsertTask` or `UpsertNotification`). The pending count badge on the `ConnectivityBar` incremented correctly with each operation.

Sync Replay

On deactivating the manual offline flag, the `OfflineSyncService` detected the transition from `IsEffectivelyOnline = false` to `true`, waited the configured 1500 ms stabilisation delay, and began the replay pass. SignalR events (`SyncStarted`, `SyncProgress`, `SyncComplete`) were observed in the browser console and drove the animated progress bar in the `ConnectivityBar`. After the pass completed, the `SyncOutboxEntry` rows were promoted to `Synced` and the corresponding documents were present in the MongoDB `tasks` collection with the correct field values.

BulkSync on Startup

A fresh user account was created with the application offline (no MongoDB connection). Twenty tasks were created. When the application was restarted with the MongoDB connection string configured and the server reachable, `BulkSyncStartupService` detected an online MongoDB within the 90-second window and performed the bulk upsert. All twenty documents appeared in the MongoDB `tasks` collection within approximately 2 seconds of the service completing its online check.

Conflict Resolution

Conflict resolution follows a last-write-wins strategy keyed on `UpdatedAt`. A task was created offline on instance A and the same task's title was edited offline on instance B (simulated by directly modifying the SQLite database). When both instances came online, the instance that synced later overwrote the earlier document in MongoDB, producing a consistent final state. This behaviour is consistent with the design rationale described in Section 2.3.

4.7 AI Chatbot

The AI chatbot was tested in all three operational modes.

General Mode

Using `mistral-small-latest`, the assistant answered questions about task prioritisation strategies and meeting scheduling best practices. Streamed responses began appearing in the UI within 400 ms of submission (measured from click to first token rendered), confirming that the SSE pipeline introduces negligible overhead beyond the Mistral API's own response latency. Conversation history was preserved across page navigation; returning to a previous conversation and sending a follow-up message confirmed that the model received the prior context correctly (evidenced by pronoun resolution across turns).

Coder Mode

Using `codestral-latest`, the assistant produced syntactically valid C# and TypeScript code samples when asked to generate a service class and a React hook. The system prompt's instruction to explain algorithmic choices was reflected in the model's responses, which included comments and rationale alongside the code. The `react-markdown` renderer with the `remark-gfm` plugin correctly formatted the code blocks with monospace styling and language labels.

Scanner Mode

Using `mistral-ocr-latest`, a one-page PDF containing a structured table was uploaded via the paperclip attachment button. The model returned the table content in Markdown format with columns correctly identified. A PNG image containing a handwritten-style label was also tested; the model extracted the text with minor errors on ambiguous glyphs.

Conversation Management

AI-generated titles were verified: the first message of each new conversation triggered a `GenerateTitleAs` call and the resulting title (typically 4–8 words) appeared in the conversation list sidebar within one second. Titles were stable across subsequent messages in the same conversation. The conversation list correctly sorted entries by `UpdatedAt` descending.

4.8 Calendar and Dashboard

Dashboard

The dashboard renders five metric cards: Total Active Tasks, In Progress, Projects, Team Members, and a Task Trend indicator (percentage change in task count over the preceding seven days). A *Due This Week* panel lists the tasks whose due date falls within the next seven days, sorted by due date ascending. A *Recent Activity* feed shows the ten most recently created or updated tasks, with relative timestamps. An embedded calendar widget in the lower section of the dashboard mirrors the weekly view from the full calendar page.

The dashboard statistics were verified against the SQLite database contents: adding three tasks raised the *Total Active Tasks* count by three; completing tasks moved them out of the active count.

Calendar Events

Calendar events are distinct from tasks: they carry a `StartTime`, `EndTime`, an optional `Colour` hex code, and an optional `MeetingLink`. Full CRUD operations were verified through the `CalendarController`. Events appear as coloured blocks in the weekly time-grid view alongside task items. A conflict check (two events occupying overlapping time slots) was handled at the UI level by rendering events in adjacent lanes.

4.9 Notifications and Settings

Notifications

The notification panel was tested for all fifteen defined notification types. Notifications arrive via SignalR in real time; the unread badge on the bell icon increments without a page reload. Clicking *Mark all as read* sends a `PUT /api/notifications/mark-all-read` request, clears the badge, and marks every row in the `Notification` table as `IsRead = true`. The `ActionUrl` field was verified: clicking a *TaskAssigned* notification navigated the user to the correct task detail view.

`DueDateWarningService` was tested by setting a task's due date to one hour in the future. After the next polling cycle (within 1 minute), a *DueDateApproaching* notification appeared in the notification panel and a duplicate-suppression check confirmed that the notification was not re-generated on subsequent polling cycles within the same session.

Reminders

User-created reminders were tested by setting a `FireAt` timestamp one minute into the future. `ReminderProcessorService` detected the reminder on its next polling tick, sent a `ReminderFired` notification, and set `HasFired = true` to prevent re-firing.

Settings

The settings page was divided into four sections: *Profile*, *Security*, *Notifications*, and *Appearance*. Profile updates (name, avatar URL, company, country, phone, timezone) were persisted to `AppUser` immediately and reflected on the settings page after save. The password change flow correctly required the current password before accepting a new one, and rejected mismatched confirmation values with a client-side validation error. Notification preference toggles (email notifications on/off per event type) were stored in `AppUser` and respected by the `DueDateWarningService` and `ReminderProcessorService` when deciding whether to dispatch email alerts.

4.10 Cross-Platform Distribution

The packaged desktop application was tested on three operating systems.

Windows (NSIS installer). The installer was built and executed on Windows 11 (x64). The `oneClick: false` NSIS configuration presented the user with an installation directory picker and optional shortcuts for the Desktop and Start Menu. After installation, launching the application from the Start Menu shortcut spawned the Electron process, which in turn started

the embedded .NET backend. The `TASKFLOW_BACKEND_READY` signal was received within approximately 3 seconds on the test machine (Intel Core i7, 16 GB RAM, NVMe SSD), after which the React UI loaded in the Electron window. The same installer was tested on Windows 10 (version 22H2) with identical results.

macOS (DMG). The DMG was mounted on macOS 14 Sonoma (Apple Silicon, M2 chip). Because the binary was not notarised, macOS Gatekeeper required the user to right-click and select *Open* on the first launch. After this one-time override, subsequent launches proceeded without warnings. The application operated correctly; the Rosetta 2 translation layer was not invoked as the build targeted `osx-arm64` directly.

Linux (AppImage). The AppImage was executed on Ubuntu 22.04 (x64) after marking it executable with `chmod +x`. No system-level dependencies needed to be installed. The application launched, authenticated successfully against the embedded backend, and performed offline task creation and synchronisation without errors. The DEB package was also installed via `dpkg -i` on the same machine and launched from the application menu.

In all three environments the application started without requiring the user to install Node.js, .NET, or any other runtime, satisfying NFR-03 (Cross-Platform Distribution).

4.11 Performance Observations

Informal timing measurements were taken during testing to assess NFR-04 (Responsiveness). All measurements were made on the Windows 11 test machine described above.

Table 4.2: Informal response-time observations for key operations.

Operation	Observed latency
Application cold start (Electron to UI ready)	≈ 3 s
Login (JWT issue + MongoDB pull)	<500 ms
Task list load (100 tasks, SQLite)	<100 ms
Task create (SQLite write + SignalR event)	<80 ms
First AI token rendered (General mode)	≈ 400 ms
Offline-to-online sync replay (20 entries)	≈ 1.8 s
Direct message delivery (sender to recipient)	<150 ms

All interactive operations satisfied the 200 ms threshold defined in NFR-04, with the exception of application cold start and AI first-token latency, which are dominated by process initialisation and network round-trip time respectively and are not considered interactive UI responses.

Chapter 5

Conclusion

This chapter summarises the work carried out, evaluates the extent to which the original objectives were achieved, reflects on the technical decisions made during the project, and acknowledges the limitations of the delivered system.

5.1 Summary of Work

Task Flow was conceived in response to a concrete problem: existing task management tools are web-first services that become inaccessible or functionally degraded when network connectivity is lost, yet mobile or remote workers routinely face exactly those conditions. The objective was to build a self-contained desktop application that delivers the full feature set of a modern task management platform while remaining fully operational without an internet connection and automatically synchronising data when connectivity is restored.

The six objectives established in Section 1.3 were all met:

1. **Cross-platform desktop application.** An Electron 41 shell embeds the ASP.NET Core 10 backend and the React 19 frontend in a single redistributable package. Installers were produced and tested for Windows (NSIS), macOS (DMG), and Linux (AppImage and DEB). No prior installation of Node.js, .NET, or any other runtime is required on the target machine (Section 4.10).
2. **Offline-first task management.** Every write operation produces both an immediate SQLite record and a `SyncOutboxEntry` in the outbox queue. The `OfflineSyncService` replays the queue when connectivity is restored and `BulkSyncStartupService` handles the case in which the user was offline during the preceding session. Twenty-entry offline-then-sync scenarios were verified without data loss (Section 4.6).
3. **Multiple task visualisation modes.** Five views — Default, Kanban, Table, Gantt, and

Calendar — each present the task dataset through a different organisational lens. The Kanban drag-and-drop, the Gantt year timeline, and the Calendar weekly time-grid were all implemented and tested on at least two operating systems (Section 4.4).

4. **Team collaboration and real-time communication.** Project and team invitations flow through MongoDB as a relay layer. Direct messages and group chats are persisted in SQLite and delivered in real time via a SignalR hub. File attachments up to 50 MB are supported. Notification delivery was verified for all fifteen event types (Sections 4.5 and 4.9).
5. **Integrated AI assistant.** The chatbot integrates the Mistral API in three modes: general productivity guidance, code generation and explanation, and document OCR and summarisation. Responses are streamed token-by-token over Server-Sent Events with a median first-token latency of approximately 400 ms in the general mode (Section 4.7).
6. **Maintainable, extensible codebase.** The backend follows a layered architecture (Controllers, Services, Repositories, Data) enforced by constructor injection via the ASP.NET Core DI container. The `ISyncableEntity` abstraction decouples the sync engine from individual entity types, allowing new syncable entities to be added by implementing the interface and registering a corresponding repository. AutoMapper profiles and FluentValidation validators are co-located with their respective DTOs and entities.

5.2 Technical Contributions

The project produced four technical contributions that go beyond assembling existing libraries.

Outbox-pattern offline sync engine for a desktop .NET application. While the transactional outbox pattern is well documented in distributed-systems literature, applying it within a single-user SQLite-backed desktop application required adapting the pattern to a process-local context. The `ConnectivityService` acts as the circuit breaker; the `OfflineSyncService` provides the consumer loop; and the `ISyncableEntity` interface defines the contract that entities must satisfy to participate in sync. The result is a lightweight, dependency-free sync engine that requires no message broker and adds less than one millisecond of overhead per write operation on the test machine.

Dual-database architecture with a shared entity abstraction. Combining an embedded relational database (SQLite) as the primary store with a cloud-hosted document database (MongoDB) as a relay layer is an uncommon design for a desktop application. The `ISyncableEntity` interface exposes a `ToMongoDocument()` method that produces a BSON-compatible dictionary, keeping the transformation logic on the entity itself rather than scattered across service

classes. This design made it straightforward to add or modify syncable entities throughout development without changing the sync engine.

SSE-streamed multi-modal AI chatbot in a desktop context. Integrating a streaming HTTP response (SSE) inside an Electron-hosted application required ensuring that the Electron main process did not buffer or terminate the long-lived HTTP connection before all tokens were delivered. The solution — issuing the SSE request directly from the React renderer process to the local ASP.NET Core server — bypassed the main process entirely and delivered correct streaming behaviour without any special Electron configuration.

Self-contained cross-platform packaging with embedded runtime. Bundling an ASP.NET Core server, a React frontend, and an Electron shell into a single installer that starts all three processes transparently on Windows, macOS, and Linux required careful orchestration of the Electron main process. The stdout-signal protocol (`TASKFLOW_BACKEND_READY:{address}`) decouples the frontend load from the backend start time, allowing the UI to display a splash state until the server is confirmed ready regardless of machine speed.

5.3 Lessons Learned

Several broader lessons emerged from the design and implementation process.

Offline-first requires deliberate UI design, not just backend changes. It is not sufficient to enqueue writes; users need visible confirmation that their actions have been recorded and will eventually reach the server. The `ConnectivityBar` with its pending-count badge, colour-coded connectivity states, and animated sync progress bar was as important to the user experience as the outbox queue itself. Implementing this required defining a clear contract between the sync service and the React component so that both could react to the same state transitions.

Choosing the right conflict resolution strategy early saves significant rework. The decision to use last-write-wins based on `UpdatedAt` was made early and shaped the entity interface, the outbox entry schema, and the MongoDB upsert logic. Had this decision been deferred, adding a timestamp field and revision counter to every entity late in the project would have required a database migration on potentially live data.

SignalR group-per-user is a reliable notification pattern. Joining each authenticated connection to a group named after the user's ID (`Groups.AddToGroupAsync(Context.ConnectionId, userId)`) meant that notifications could be dispatched to any service class by calling `_hubContext.Clients.groupName.SendAsync(...)` without needing to track connection IDs. This pattern handled reconnection transparently: a reconnecting client re-joined its group and received any notifications dispatched after reconnection without any server-side state changes.

Electron child-process IPC via stdout is reliable but fragile. The stdout-signal approach works well in production builds but can be disrupted during development when the backend writes diagnostic output before the ready signal. Filtering the stdout stream with a prefix (`TASKFLOW_BACKEND_READY:`) rather than reading the first line proved more robust, but a dedicated named-pipe or socket protocol would be a more principled solution.

5.4 Limitations

The following limitations were identified during development and testing.

Last-write-wins conflict resolution. The synchronisation engine uses a simple last-write-wins strategy based on the `UpdatedAt` timestamp. In scenarios where two users edit the same shared task concurrently while offline, the later sync will silently overwrite the earlier one. No merge dialogue or conflict notification is presented to the user. For the single-user scenario (one user across multiple devices) this is acceptable; for multi-user collaborative editing it may result in lost updates.

Sub-task backend not implemented. The Table View renders a placeholder sub-task row beneath each task, and the task entity schema includes a comment referring to sub-tasks. However, no `SubTask` entity, migration, service, or API endpoint was created. The feature exists at the UI layer only and is not persisted. This is the largest gap between the stated scope and the delivered system.

Mistral API key is hardcoded. The API key used to authenticate requests to the Mistral API is stored as a compile-time constant in `MistralChatService.cs`. This violates the principle of separating secrets from source code and means that rotating the key requires a recompile and redistribution of the application. The recommended remedy — reading the key from the `TASKFLOW_MISTRAL_KEY` environment variable at startup and falling back to a configuration value — was identified but not implemented within the project timeline.

No automated end-to-end test suite. Testing was performed manually as described in Section 3.7. While the architecture is amenable to unit testing (stateless services, injected repositories), no unit tests, integration tests, or end-to-end tests were written. The absence of an automated regression suite means that confidence in correctness relies entirely on manual verification.

macOS Gatekeeper requires a manual override on first launch. Because the macOS distribution was not submitted to Apple for notarisation, users on macOS are required to right-click and select *Open* on the first launch. This is a distribution-process limitation rather than a functional defect but represents a friction point for non-technical users.

Chapter 6

Future Work

The delivered system meets all stated objectives and is fully functional as a cross-platform desktop application. Nevertheless, a number of directions for improvement and extension were identified during development. These are grouped into short-term improvements that address known gaps in the current implementation and longer-term enhancements that would expand the scope of the system.

6.1 Short-Term Improvements

Sub-task backend implementation. The Table View renders a placeholder sub-task row for each task, but no corresponding backend exists. Implementing sub-tasks would require adding a `SubTask` entity with a foreign key to `TaskItem`, a database migration, a `SubTasksController`, and a `SubTaskRepository`. The `ISyncableEntity` interface would then be applied to `SubTask` so that sub-tasks participate in the offline synchronisation pipeline automatically. The UI work — expanding the Table View row and exposing CRUD controls — is largely already scaffolded. This is the highest-priority item because it is the largest gap between the stated scope and the delivered system.

Mistral API key security. As noted in Section 5.4, the Mistral API key is currently a compile-time constant. The immediate remedy is to read the key from an environment variable (`TASKFLOW_MISTRAL`) at startup, falling back to the `appsettings.json` configuration value, and ultimately defaulting to the embedded constant only as a last resort. In a production distribution the key should be provisioned by the installer or entered by the user on first launch and stored in the operating system's credential store (Windows Credential Manager, macOS Keychain, or `libsecret` on Linux).

Automated test suite. The absence of automated tests is the most significant quality risk in the codebase. A practical test suite would consist of three layers:

1. **Unit tests** (xUnit) for pure service methods and the sync engine logic, using mocked repository interfaces.
2. **API integration tests** using `WebApplicationFactory<Program>` to spin up an in-memory ASP.NET Core server backed by an in-memory SQLite database, covering all controller endpoints.
3. **End-to-end tests** using Playwright to exercise the packaged Electron application, verifying that key user journeys (login, create task, go offline, create task, go online, confirm sync) complete correctly.

Adding this suite would allow regressions to be detected automatically and would provide the confidence required to accelerate future feature development.

Per-field conflict resolution. The current last-write-wins strategy resolves sync conflicts at the document level: the document with the later `UpdatedAt` timestamp overwrites the earlier one entirely. A more granular approach would track a per-field version vector, merging non-conflicting field changes and presenting a conflict dialogue only when the same field was edited concurrently. This would be a meaningful improvement for multi-device users who edit different properties of the same task on different machines while offline.

macOS notarisation. Submitting the macOS distribution to Apple’s notarisation service would eliminate the Gatekeeper manual-override step described in Section 4.10. Notarisation requires an Apple Developer Programme membership and a CI pipeline step that submits the signed DMG to Apple and waits for approval, but the result is a significantly smoother first-launch experience for macOS users.

6.2 Long-Term Enhancements

Mobile companion application. A React Native application sharing the MongoDB relay layer would allow users to view and update tasks on iOS and Android devices. Because the MongoDB layer already stores a current snapshot of every syncable entity, the mobile client would not require an additional backend service: it could connect directly to MongoDB (via Atlas Data API or a lightweight proxy) and write changes that the desktop client would pick up on its next sync cycle.

AI-powered task creation from natural language. The existing Mistral integration could be extended to accept a free-text description such as “*Set up a team meeting next Thursday at 10*”

AM to review the sprint” and return a structured `CreateTaskDto` or `CreateCalendarEventDto` payload that is immediately submitted to the appropriate API endpoint. This would reduce the number of form fields a user must complete for routine task creation and leverage the AI assistant for a workflow-acceleration purpose beyond question answering.

CRDT-based synchronisation. Replacing last-write-wins with a Conflict-free Replicated Data Type (CRDT) would allow concurrent edits from multiple devices to be merged deterministically without data loss. A suitable approach for the task entity would be to model each editable field as a Last-Write-Wins Register (LWW-Register) with a logical clock, producing merge results that are commutative, associative, and idempotent. This would require replacing the current `UpdatedAt`-based merge in `UserDataSyncService` with a per-field CRDT merge function, and storing the logical clock alongside each field in the MongoDB document.

Analytics and reporting dashboard. The current dashboard provides aggregate counts and a seven-day task trend. A richer analytics view would include burndown charts, task completion velocity per team member, average task age by priority, and project-level progress bars. These metrics could be computed with MongoDB aggregation pipelines over the relay collection, avoiding additional load on the local SQLite database.

Third-party integration plugins. A plugin API that allows third-party integrations to be registered as background services would extend the application’s reach. High-value initial integrations would include bidirectional synchronisation with GitHub Issues (mapping issues to tasks), Google Calendar (mapping `CalendarEvents` to Google Calendar entries), and Slack (posting task assignment and due-date notifications to a channel). Each plugin would implement a common `IIntegrationPlugin` interface, keeping the core application decoupled from external service details.

Cloud-hosted multi-tenant mode. The current architecture is single-user and desktop-only. A future SaaS variant would replace the embedded Electron shell with a web-hosted frontend, the per-user SQLite database with a per-organisation PostgreSQL instance managed by Entity Framework Core, and the MongoDB relay with a shared cluster. The ASP.NET Core service layer would require minimal changes because its business logic is already decoupled from the storage layer via repository interfaces; the main effort would be in the authentication layer (multi-tenant JWT issuance, organisation scoping on all queries) and the deployment pipeline (containerisation, orchestration, secrets management).

Appendix

Appendix A

API Reference

Table A.1 lists all REST API controllers, their base routes, and the primary endpoints they expose. All routes are prefixed with `/api`. Every endpoint except the authentication endpoints and the connectivity check requires a valid JWT Bearer token in the `Authorization` header.

Table A.1: REST API controllers and primary endpoints.

Controller	Base route	Key endpoints
AuthController	/api/auth	POST /register, POST /login, POST /forgot-password, POST /reset-password
TasksController	/api/tasks	GET, POST /, GET, PUT, DELETE /{id}, PATCH /{id}/status, PATCH /{id}/star, GET /search
ProjectsController	/api/projects	GET, POST /, GET, PUT, DELETE /{id}, POST /{id}/invite, GET /{id}/members
TeamsController	/api/teams	GET, POST /, GET, PUT, DELETE /{id}, POST /{id}/invite, POST /invitations/{id}/accept
MessagesController	/api/messages	GET /conversations, GET /{userId}, POST /, DELETE /conversation/{userId}
GroupChatsController	/api/groupchats	GET, POST /, GET, PUT, DELETE /{id}, POST /{id}/members, POST /{id}/messages
NotificationsController	/api/notifications	GET /, PUT /{id}/read, PUT /mark-all-read, DELETE /{id}
CalendarEventsController	/api/calendarevents	GET, POST /, GET, PUT, DELETE /{id}
ChatbotController	/api/chatbot	GET /conversations, POST /conversations, GET /conversations/{id}/messages, POST /conversations/{id}/messages, POST /scan
DashboardController	/api/dashboard	GET /stats, GET /due-this-week, GET /recent-activity
TaskCommentsController	/api/tasks/{taskId}/comments	GET, POST, DELETE /{id}
SettingsController	/api/settings	GET, PUT /profile, PUT /password
ConnectivityController	/api/connectivity	GET /ping, POST /bulk-sync
DevController	/api/dev	GET /db-info (development environment only)

Appendix B

Database Schema

SQLite Entities

Table B.1 lists all eighteen SQLite entities managed by Entity Framework Core, together with their primary key type, key fields, and any sync participation.

Table B.1: SQLite entity summary.

Entity	PK		Key fields
AppUser	int	Id	FirstName, LastName, Email, PasswordHash, AvatarUrl, Company, Country, Phone, Timezone, ResetToken, ResetTokenExpiry
TaskItem	int	Id	Title, Description, Status, Priority, DueDate, IsStarred, ProjectId, AssigneeId, SyncId, UpdatedAt, IsSynced
Project	int	Id	Name, Description, Color, OwnerId, IsStarred, SyncId, UpdatedAt, IsSynced
ProjectMember	int	Id	ProjectId, UserId, Role, JoinedAt
Team	int	Id	Name, Description, OwnerId, CreatedAt
TeamMember	int	Id	TeamId, UserId, Role, JoinedAt
Message	int	Id	SenderId, ReceiverId, Body, IsRead, SentAt, AttachmentUrl, IsDeletedBySender, IsDeletedByReceiver, IsEdited
GroupChat	int	Id	Name, CreatedByUserId, CreatedAt; child: GroupChatMember (GroupChatId, UserId, JoinedAt); child: GroupChatMessage (GroupChatId, SenderId, Body)
Notification	int	Id	UserId, Title, Message, NotificationType, IsRead, ActionUrl, RelatedTaskId, SyncId, UpdatedAt
CalendarEvent	int	Id	OwnerId, Title, Description, StartAt, EndAt, Color, MeetingLink
ChatbotConversation	int	Id	UserId, Title, Mode, CreatedAt, UpdatedAt
ChatbotMessage	int	Id	ConversationId, Role, Content, CreatedAt
Reminder	int	Id	TaskId, UserId, FireAt, NotifyEmail, NotifyInApp, HasFired, FiredAt, SyncId
SyncOutboxEntry	int	Id	OperationId, OperationName, PayloadJson, Status (Pending/Processing/Synced/Failed), Attempts, LastAttemptAt, ErrorMessage
TaskComment	int	Id	TaskId, AuthorId, Body, CreatedAt
LocalInvitation	int	Id	InviteeId, InviterId, TeamId or ProjectId, Status, CreatedAt
LocalTeamMember	int	Id	TeamId, UserId, Role, JoinedAt (local mirror of MongoDB record)
ISyncableEntity			Interface only: SyncId, UpdatedAt, IsSynced, ToMongoDocument()

MongoDB Collections

Table B.2 lists the five MongoDB collections used as the relay layer.

Table B.2: MongoDB collection descriptions.

Collection	Description and key fields
tasks	Mirror of <code>TaskItem</code> . Documents keyed by <code>syncId</code> (GUID). Fields: <code>syncId</code> , <code>title</code> , <code>description</code> , <code>status</code> , <code>priority</code> , <code>dueDate</code> , <code>ownerId</code> , <code>projectId</code> , <code>isStarred</code> , <code>updatedAt</code> .
notifications	Mirror of <code>Notification</code> . Fields: <code>syncId</code> , <code>userId</code> , <code>title</code> , <code>message</code> , <code>notificationType</code> , <code>isRead</code> , <code>actionUrl</code> , <code>relatedTaskId</code> , <code>updatedAt</code> .
team_invitations	Cross-device invitation relay. Fields: <code>inviterEmail</code> , <code>inviteeEmail</code> , <code>teamId</code> , <code>projectId</code> , <code>teamName</code> , <code>status</code> (Pending/Accepted/Declined), <code>createdAt</code> .
team_members	Mirror of accepted team memberships. Fields: <code>teamId</code> , <code>userId</code> , <code>userEmail</code> , <code>role</code> , <code>joinedAt</code> .
user_presence	Optional online-status documents. Fields: <code>userId</code> , <code>lastSeenAt</code> , <code>isOnline</code> .

Appendix C

Configuration Reference

appsettings.json Structure

The application is configured via `appsettings.json` (overridden by `appsettings.Development.json` in development). Table C.1 describes each configuration section.

Table C.1: appsettings.json section reference.

Section / key	Description
DatabaseProvider	"SQLite" (default) or "SqlServer". Selects the EF Core provider at startup.
ConnectionStrings	Default connection string, e.g. Data Source=taskflow.db. Relative paths are resolved against the executable directory.
MongoDB.Connection	Full MongoDB connection string including credentials. Leave empty to run in SQLite-only mode without sync.
Jwt.Key	HMAC-SHA256 signing key (minimum 32 characters). If empty, the application generates a random 32-byte key at startup (stored only in memory; invalidated on restart). Set TASKFLOW_JWT_KEY environment variable to override.
Jwt.Issuer / .Audience	Token issuer and audience strings. Defaults: TaskFlow / TaskFlowApp.
Jwt.ExpiresInMinutes	Token lifetime in minutes. Default: 1440 (24 hours).
Mistral.Model	Model ID for the General chatbot mode. Default: mistral-small-latest.
Mistral.CoderModel	Model ID for the Coder mode. Default: codestral-latest.
Mistral.OcrModel	Model ID for the Scanner (OCR) mode. Default: mistral-ocr-latest.
Mistral.HistoryWindow	Number of prior messages included in the Mistral context. Default: 20.
EmailSettings.*	SMTP configuration for MailKit. Smtphost, Smtpport (587), UseSsl (false for STARTTLS), SenderEmail, SenderName, Username, Password.
Kestrel.Endpoints	http://*:27.0.0.1:0 — binds to a random available port. The chosen port is printed to stdout as TASKFLOW_BACKEND_READY:{address}.

Environment Variables

Table C.2 lists the environment variables recognised by the application at startup.

Table C.2: Supported environment variables.

Variable	Effect
TASKFLOW_JWT_KEY	Overrides <code>Jwt:Key</code> in <code>appsettings.json</code> . Must be at least 32 characters. Set by the Electron main process to a randomly generated 32-byte hex value on each launch.
ASPNETCORE_ENVIRONMENT	Standard ASP.NET Core environment selector. Set to <code>Development</code> to enable Swagger UI and verbose logging.
ASPNETCORE_URLS	Overrides Kestrel endpoint binding when set. Normally not set; the <code>Kestrel</code> config section is used instead.

electron-builder.json Summary

Table C.3 summarises the key `electron-builder.json` settings used to produce the cross-platform installers.

Table C.3: Key `electron-builder.json` configuration.

Key	Value / description
<code>appId</code>	<code>com.taskflow.desktop</code>
<code>productName</code>	Task Flow
<code>win.target</code>	NSIS (Windows installer)
<code>nsis.oneClick</code>	false — user selects installation directory
<code>nsis.createDesktopShortcut</code>	true
<code>nsis.createStartMenuShortcut</code>	true
<code>mac.target</code>	DMG + ZIP
<code>linux.target</code>	AppImage + DEB
<code>files</code>	Bundled assets include <code>public/</code> , <code>wwwroot/</code> , the compiled .NET binary under <code>bin/Release/</code> , and <code>node_modules/</code>
<code>extraResources</code>	.NET self-contained runtime and backend binary

Bibliography

- [1] David Allen. *Getting Things Done: The Art of Stress-Free Productivity*. Penguin Books, 2001.
- [2] Albert Q. Jiang, Alexandre Sablayrolles, Arthur Mensch, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Florian Bressand, Gianna Lengyel, Guillaume Lample, Lucile Saulnier, et al. Mistral 7B. arXiv preprint arXiv:2310.06825, 2023. Available: <https://arxiv.org/abs/2310.06825>.
- [3] Marc Shapiro, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. Conflict-Free Replicated Data Types. In *Proceedings of the 13th International Symposium on Stabilization, Safety, and Security of Distributed Systems (SSS)*, pages 386–400. Springer, 2011.
- [4] David J. Anderson. *Kanban: Successful Evolutionary Change for Your Technology Business*. Blue Hole Press, 2010.
- [5] Electron Contributors. Electron Documentation. <https://www.electronjs.org/docs>, 2024. Accessed 2024.
- [6] Martin Kleppmann. *Designing Data-Intensive Applications*. O’Reilly Media, 2017.
- [7] Eric Brewer. CAP Twelve Years Later: How the “Rules” Have Changed. *Computer*, 45(2):23–29, 2012.
- [8] Martin Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.
- [9] Martin Kleppmann and Alastair R. Beresford. A Conflict-Free Replicated JSON Datatype. *IEEE Transactions on Parallel and Distributed Systems*, 28(10):2733–2746, 2017.
- [10] Kevin P. Gaffney, Martin Prammer, Larry Brasfield, D. Richard Hipp, Dan Kennedy, and Jignesh M. Patel. SQLite: Past, Present, and Future. *Proceedings of the VLDB Endowment*, 15(12):3535–3547, 2022.
- [11] Kristina Chodorow. *MongoDB: The Definitive Guide*. O’Reilly Media, 2nd edition, 2013.

- [12] Microsoft. Entity Framework Core Documentation. <https://docs.microsoft.com/en-us/ef/core>, 2024. Accessed 2024.
- [13] I. Fette and A. Melnikov. The WebSocket Protocol. RFC 6455, Internet Engineering Task Force, December 2011. Available: <https://www.rfc-editor.org/rfc/rfc6455>.
- [14] Microsoft. ASP.NET Core SignalR Documentation. <https://docs.microsoft.com/en-us/aspnet/core/signalr>, 2024. Accessed 2024.
- [15] M. Jones, J. Bradley, and N. Sakimura. JSON Web Token (JWT). RFC 7519, Internet Engineering Task Force, May 2015. ISSN 2070-1721. Available: <https://www.rfc-editor.org/rfc/rfc7519>.
- [16] Niels Provos and David Mazières. A Future-Adaptable Password Scheme. In *Proceedings of the 1999 USENIX Annual Technical Conference*, pages 32–32. USENIX Association, 1999.
- [17] OWASP Foundation. OWASP Top 10 Application Security Risks – 2021. <https://owasp.org/Top10/>, 2021. Accessed 2024.
- [18] Ravi S. Sandhu, Edward J. Coyne, Hal L. Feinstein, and Charles E. Youman. Role-Based Access Control Models. *Computer*, 29(2):38–47, 1996.
- [19] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention Is All You Need. *Advances in Neural Information Processing Systems*, 30, 2017.
- [20] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language Models are Few-Shot Learners. *Advances in Neural Information Processing Systems*, 33:1877–1901, 2020.
- [21] James M. Wilson. Gantt Charts: A Centenary Appreciation. *European Journal of Operational Research*, 149(2):430–437, 2003.
- [22] Microsoft. ASP.NET Core Documentation. <https://docs.microsoft.com/en-us/aspnet/core>, 2024. Accessed 2024.